PRODYNA Cloud Engineering Task

Infrastruktur als Code · Azure PaaS · GitHub Actions

19.03.2026 Robin Brandoni

Agenda

01 Aufgabenstellung

02 Solution Design

03 Infrastructure as Code

04 Repository-Struktur

05 CI/CD Pipeline

06 Deployment- & Git-Strategie

07 Fazit

Aufgabenstellung

Was war zu tun?

Ziel

- Cloud-Umgebung für eine
- Single Page Application
- Automatisierte CI/CD Pipeline
- Cloud-native Technologien

Tech-Stack

- Vue.js – Frontend
- Node.js – Backend
- MongoDB – NoSQL-Datenbank

Anforderungen

- Microsoft Azure als Hyperscaler
- PaaS bevorzugt
- IaC + vollautom. Deployments

Solution Design

Gewählte Architektur auf Azure

Gewählte Lösung

Azure Static Web Apps
Frontend

Azure App Service
Backend

Azure Cosmos DB
Datenbank (MongoDB API)

Azure Key Vault
Secrets & Konfiguration

Azure Monitor
Monitoring & Insights

Vorteile PaaS

- Kein operativer Overhead
- Automatische Skalierung
- Native Azure Integration

Alternative: AKS

- Höhere Flexibilität & Portabilität
- Mehr Kontrolle über Container
- Deutlich höherer Betriebsaufwand

Infrastructure as Code

Warum Terraform?

Versioniert

Infrastruktur im Git-Repo

Reproduzierbar

Idempotente Deployments

Standard-Tool

Weit verbreitet & bewährt

CI/CD-ready

Native Pipeline-Integration

Terraform-Umgebungen

DEV

Entwicklungsumgebung

STAGING

Test- & Integrationsumgebung

PROD

Produktionsumgebung

Repository-Struktur

Monorepo – alles an einem Ort, klar strukturiert

architecture/ — DOKU

Architekturdiagramme & Konzepte

terraform/ — IaC

Infrastruktur-Definition (IaC)

pipelines/ — CI/CD

GitHub Actions Workflows

presentation/ — DOKU

Präsentation & Dokumentation

Gemeinsame Codebasis

Kein Sync-Aufwand zwischen Repos

Vollständige Historie

Infra & Code gemeinsam versioniert

Schnell erweiterbar

Neue Komponenten einfach hinzufügen

CI/CD Pipeline

Vom Code-Commit bis zum Deployment – vollautomatisch

CI

Continuous Integration

1. Checkout

Code auschecken

2. Install

npm install

3. Build

Vue.js kompilieren

4. Test

Unit Tests Node.js

5. Build Application

erstellte Applikation

CD

Continuous Deployment

1. TF Init

Provider initialisieren

2. TF Plan

Änderungen prüfen

3. Approval

Manuelles Freigabe-Gate

4. TF Apply

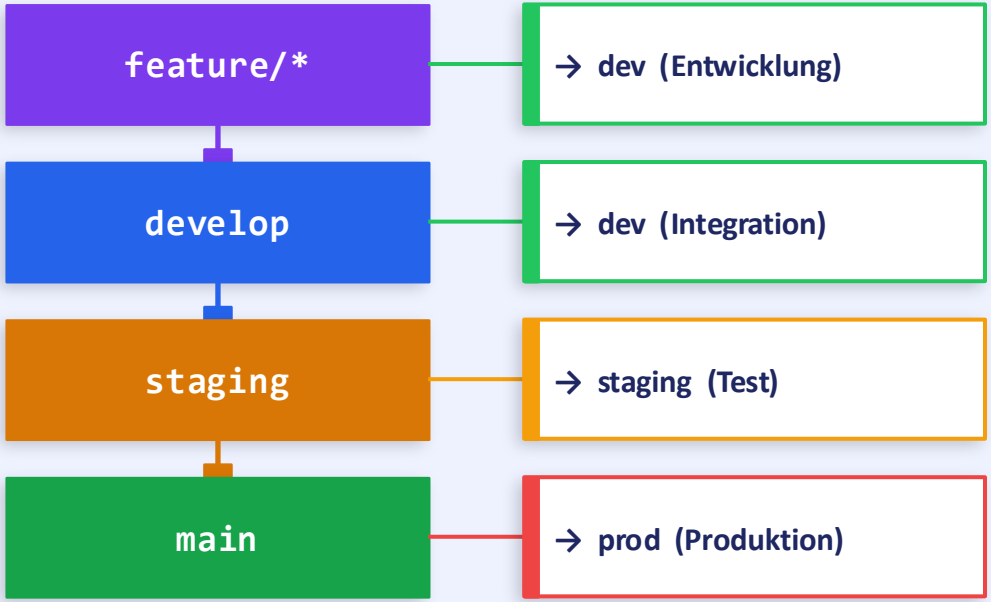
Infrastruktur anwenden

5. Deploy App

App auf Azure deployen

Deployment- & Git-Strategie

Branching-Modell & Umgebungsstrategie



Absicherung Produktion

- Protected main branch
- Pull Requests verpflichtend
- CI Pipeline muss grün sein
- Manuelles Freigabe-Gate
- Keine direkten Pushes auf main

Fazit

Was wurde erreicht?

Cloud-Native

Vollständig auf
Azure PaaS

Automatisiert

Zero-Touch Deployment
von Commit bis Prod

Abgesichert

Protected Branches
& manuelle Gates

Skalierbarkeit

Geringer Betriebsaufwand

Schnelle Release-Zyklen